

Self-Learning: Distributed Transactions, TCL & Performance Tuning — DBMS

Definition: Distributed Transactions execute across multiple independent database nodes connected over a network, managed using the **Two-Phase Commit (2PC)** protocol.

1. Detailed Technical Explanation

1. Two-Phase Commit (2PC) Protocol

Coordinates atomic commit/abort decisions across multiple distributed node participants.

COORDINATOR	PARTICIPANTS (Nodes 1, 2, 3)
----- Phase 1: PREPARE Message ----->	
<----- VOTE_COMMIT / VOTE_ABORT -----	(Check local log & resources)
(If all VOTE_COMMIT):	
----- Phase 2: GLOBAL_COMMIT Message ----->	
<----- ACKNOWLEDGEMENT -----	(Write commit to local disk)

2. Transaction Control Language (TCL) Commands

```
-- 1. Start a explicitly scoped transaction
START TRANSACTION;

-- 2. Modify data
UPDATE Accounts SET Balance = Balance - 5000 WHERE Acc_No = 101;

-- 3. Set a rollback checkpoint
SAVEPOINT sp1;

UPDATE Accounts SET Balance = Balance + 5000 WHERE Acc_No = 102;

-- 4. Rollback to Savepoint if error occurs
ROLLBACK TO SAVEPOINT sp1;

-- 5. Commit remaining updates permanently
COMMIT;
```

3. Database Performance Tuning Strategies

1. Indexing Optimization:

- Create B+ Tree indexes on frequently queried `WHERE` , `JOIN` , and `ORDER BY` columns.
- Avoid over-indexing (slows down `INSERT` , `UPDATE` , `DELETE` operations).

2. Query Refactoring:

- Avoid `SELECT *` ; specify explicit columns to reduce I/O bandwidth.
- Replace correlated subqueries with explicit `JOIN` operations.

3. Buffer Pool & Memory Tuning:

- Allocate adequate RAM to database buffer pool to maximize cache hit ratio (>95%).

4. Partitioning & Sharding:

- Range/Hash partition large tables to distribute disk I/O bottlenecks.
-

4. Quick Recall Flow

2PC -> Prepare Phase (Vote) -> Commit Phase (Global ACK) | TCL -> COMMIT, ROLLBACK, SAVEPOINT | Tuning -> B+ Tree Indexes